COMP 442
Compiler Design

Assignment 4: Semantic Analysis

Writer:
Théo Bary (ID: 40348931)

Instructor: Dr. Joey Paquet
joey.paquet@concordia.ca

Date Due: Sunday, 22 March 2026, 11:59 PM

"I certify that this submission is my original work and meets the Faculty's Expectations of Originality"

Section 1 - Semantic Rules: 2
Section 2 - Design: 3
Part I - Overall design: 3
Part II - Phases: 4
Section 3 - Use of tools: 5

Section 1 - Semantic Rules:

- ☑ 6.1 undeclared member function definition
- ☑ 6.2 undefined member function declaration
- ☑ 8.1 multiply declared class
- ☑ 8.2 multiply declared free function
- ☑ 8.3 multiply declared data member in class
- ☑ 8.4 multiply declared variable in function
- ☑ 8.5 shadowed inherited data member
- ☑ 8.6 local variable in a member function shadows a data member of its class
- ☑ 9.1 Overloaded free function
- ☑ 9.2 Overloaded member function
- ☑ 9.3 Overridden member function
- ☑ 10.1 Type error in expression
- ☑ 10.2 Type error in assignment statement
- ☑ 10.3 Type error in return statement
- ☑ 11.1 Undeclared local variable
- ☑ 11.2 Undeclared member variable
- ☑ 11.3 Undeclared member function
- ☑ 11.4 Undeclared/undefined free function
- ☑ 11.5 Undeclared class
- ☑ 12.1 Function call with wrong number of parameters
- ☑ 12.2 Function call with wrong type of parameters
- ☑ 13.1 Use of array with wrong number of dimensions
- ☑ 13.2 Array index is not an integer
- ☑ 13.3 Array parameter using wrong number of dimensions
- ☑ 14.1 Circular class dependency
- ☑ 15.1 "." operator used on non-class type

Section 2 - Design:

Part I - Overall design:

In this assignment I introduced 2 new classes and implemented one of them retro-actively in the project.

The first one is a Problems class that can store tokens/nodes, an error message, error step and error level (Info, Warning, Error). This class stores these and is able to display them in a nice way while highlighting the corresponding problematic token(s) all the way from the lexer no matter which step the compiler is at.

With it I am able to have a nice problem count for each step of the compiler and display them nicely (colored using [ANSI escape codes]() for readability):

```
[info    ] assignments/assignment4/tests/test_6_1_undeclared_member_function_definition.src: Lexed 60 tokens in    0.09ms
[info    ] assignments/assignment4/tests/test_6_1_undeclared_member_function_definition.src: Parsed in                    0.15ms          [0 info(s), 0 warning(s), 0 error(s)]
[info    ] assignments/assignment4/tests/test_6_1_undeclared_member_function_definition.src: Generated Symbol Table in    0.29ms          [0 info(s), 0 warning(s), 1 error(s)]

[error   ] assignments/assignment4/tests/test_6_1_undeclared_member_function_definition.src:13:8: 6.1 undeclared member function definition: definition provided for undeclared member function 'Shape::compute'
  13     |       Shape::compute(float x) : float
         |               ^~~~~~~
```

The second class introduced is the one doing the heavy lifting for this assignment: the SemanticAnalyzer. It starts off with a valid AST node (root) and starts by building a symbol table.

If the table is generated with no issues it then applies semantic checks (see [Section 1 - Semantic Rules]() for list). Reporting them all at once after the symbol table is done generating.

Part II - Phases:

I chose to have all the semantic checking done after generation of the symbol table for the sake of separation. It made my symbol table generation code cleaner and easier to fix.

The first checks are always done in the same order:

1. Multiple declarations (8.1, 8.2, 8.3, 8.4)
2. Function Declaration/Implementation parity (6.1, 6.2)
3. Overloads and Overrides (9.1, 9.2, 9.3)
4. Shadowing (8.5, 8.6)
5. Circular dependencies (14.1)
6. Undeclared class types (11.5)

The second phase is for everything relating to statements (I.E. return statements, assign statements, etc...) and is done by order of tokens found through traversal. It involves creating a context and checking against it for each statement encountered. Making sure types are compatible with each other and the variable/function trying to be accessed is available in the current scope.

Section 3 - Use of tools:

For this assignment I introduced a 3rd party library found on github called [tabulate](). I originally wrote my own table-to-text generator but the code ended up being very long and impossible to debug. Even though I arrived at an almost satisfying result, I could not iron out a bug related to missing column characters (|) for sub-tables.

This led me to scrap everything and use this 3rd party library. It allows me to construct a data structure using c++ objects and configure it using a [builder pattern]. It then automatically outputs a fully rendered string that I can print all at once into the desired file.

```
+-----------------------------------------------------+
| table: global                                       |
+-----------------------------------------------------+
| +-----------+-------------+--------------------+ |
| | function | bubbleSort | (int[],int):void | |
| +-----------+-------------+--------------------+ |
| +-------------------------------+                  |
| | table: ::bubbleSort           |                  |
| +-------------------------------+                  |
| | +-------+------+-------+--+ |                  |
| | | param | arr  | int[] |  | |                  |
| | +-------+------+-------+--+ |                  |
| | | param | size | int   |  | |                  |
| | +-------+------+-------+--+ |                  |
| | | local | n    | int   |  | |                  |
| | +-------+------+-------+--+ |                  |
| | | local | i    | int   |  | |                  |
| | +-------+------+-------+--+ |                  |
| | | local | j    | int   |  | |                  |
| | +-------+------+-------+--+ |                  |
| | | local | temp | int   |  | |                  |
| | +-------+------+-------+--+ |                  |
| +-------------------------------+                  |
+-----------------------------------------------------+
| +-----------+-------------+--------------------+ |
| | function | printArray | (int[],int):void | |
| +-----------+-------------+--------------------+ |
| +-------------------------------+                  |
| | table: ::printArray           |                  |
| +-------------------------------+                  |
| | +-------+------+-------+--+ |                  |
| | | param | arr  | int[] |  | |                  |
| | +-------+------+-------+--+ |                  |
| | | param | size | int   |  | |                  |
| | +-------+------+-------+--+ |                  |
| | | local | n    | int   |  | |                  |
| | +-------+------+-------+--+ |                  |
| | | local | i    | int   |  | |                  |
| | +-------+------+-------+--+ |                  |
| +-------------------------------+                  |
+-----------------------------------------------------+
| +-----------+------+---------+                     |
| | function | main | ():void |                     |
| +-----------+------+---------+                     |
| +-------------------------------+                  |
| | table: ::main                 |                  |
| +-------------------------------+                  |
| | +-------+-----+--------+--+ |                  |
| | | local | arr | int[7] |  | |                  |
| | +-------+-----+--------+--+ |                  |
| +-------------------------------+                  |
+-----------------------------------------------------+
```